GUARDIAN AI: AI-POWERED PERSONAL SAFETY PLATFORM

PROJECT PHASE 1

Submitted by:

AKSHAY B A (LMC23CS026)

ANANDHU A (LMC23CS035)

MANU V S (LMC23CS085)

to

the APJ Abdul Kalam Technological University

in the partial fulfilment of the requirements for the award of the Degree

of

Bachelor of Technology

in

Computer Science and Engineering



Department of Computer Science and Engineering

Lourdes Matha College of Science and Technology

Lourdes Hills, Kuttichal

OCTOBER 2026

DECLARATION

We undersigned hereby declare that the project phase 1 report "Guardian AI: AI-Powered Personal Safety Platform", submitted for partial fulfilment of the requirements for the award of degree of Bachelor of Technology of the APJ Abdul Kalam Technological University, Kerala is a bonafide work done by us under supervision of Ms. Nisha O S. This submission represents our ideas in our own words and where ideas or words of others have been included; we have adequately and accurately cited and referenced the original sources. We also declare that we have adhered to ethics of academic honesty and integrity and have not misrepresented or fabricated any data or idea or fact or source in our submission. We understand that any violation of the above will be a cause for disciplinary actions by the institute and/or the University and can also evoke penal action from the sources which have thus not been properly cited or from whom proper permission has not been obtained. This report has not been previously formed the basis for the award of any degree, diploma or similar title of any other University.

Place: Thiruvananthapuram

Date : 27/10/2026

Akshay B A

Anandhu A

Manu V S

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

LOURDES MATHA COLLEGE OF SCIENCE AND TECHNOLOGY

LOURDES HILLS, KUTTICHAL

CERTIFICATE

This is to certify that the report entitled **Guardian AI: AI-Powered Personal Safety Platform** by **Akshay B A (LMC23CS026), Anandhu A (LMC23CS035), Manu V S (LMC23CS085)** to the APJ Abdul Kalam Technological University in partial fulfilment of the requirements for the award of the Degree of Bachelor of Technology in Computer Science and Engineering is a bonafide record of the project work carried out by them under my guidance and supervision. This report in any form has not been submitted to any other University or Institute for any purpose.

Ms. Nisha O S	**Ms. Ambily Jane**	**Ms. Priya Sekhar S**
Asst. Professor	**Asst. professor**	**Head of Department**
Department of CSE	**Department of CSE**	**Department of CSE**
LMCST	**LMCST**	**LMCST**
(Project Guide)	**(Project Coordinator)**	

ACKNOWLEDGEMENT

We would like to express our sincere gratitude to all those who helped us in the completion of this work. We faced many difficulties due to our lack of knowledge and experience, but these people helped us to overcome them and finally complete our idea of this project. We consider ourselves privileged to express gratitude and respect towards all those who guided us through the completion of this project. We would like to thank Dr. Priya Sekhar S, HOD of the Department of Computer Science and Engineering, for providing encouragement, constant support and guidance, which was of great help in completing this project successfully. We would also like to show our gratitude to our project guide Ms. Nisha O S for the continuous help and monitoring during the project work. We are thankful and fortunate to get constant encouragement, support and guidance from all teaching and non-teaching staff of the Department of Computer Science and Engineering who helped us in successfully completing our project work. We are also thankful to our whole class and most of all our parents, who have inspired us to face all the challenges and win all the hurdles in life.

ABSTRACT

People who travel, commute or walk alone often have no fast and reliable way to ask for help, and many existing safety apps depend on a single channel such as mobile internet, drain the battery, or raise frequent false alarms. This project proposes Guardian AI, an Android personal safety platform that gets the user's location and an alert to trusted guardians within seconds, even when the internet is poor or unavailable.

Guardian AI lets a user trigger an emergency manually through an in-app SOS button, a layered power-button and shake trigger, or a silent duress PIN. It can also detect possible emergencies automatically from phone-drop sensor patterns, low battery, route deviation during a safety journey, and missed two-way check-ins. Every trigger passes a confirmation countdown and then follows one emergency state machine. Alerts are sent through Firebase Cloud Messaging and live location when the internet is available, fall back to SMS with GPS coordinates when it is not, and are stored locally and retried if both fail. Guardians join only through an invite and explicit consent, and see the user's status, live location and incident history in a guardian dashboard.

The system adds AI features on top of the safety workflow: a transparent risk score for routes and times of day, AI-generated incident summaries for guardians, and an anonymous community hotspot map. The app is built with Kotlin, Jetpack Compose and a Clean Architecture design, uses Firebase for authentication, data, notifications and storage, and supports English, Malayalam and Hindi with 112 (India ERSS) integration. This report covers Phase 1 of the project: requirements, analysis, modules, architecture and the implementation plan.

CONTENTS

TITLE	PAGE NO.
ACKNOWLEDGEMENT	**i**
ABSTRACT	**ii**
LIST OF TABLES	**iii**
LIST OF FIGURES	**iv**
1. INTRODUCTION	**1**
1.1 Objectives	1
2. LITERATURE REVIEW	**2**
2.1 Existing Solutions	2
2.2 Limitations of Current Solutions	2
2.3 Proposed Solutions	3
3. SYSTEM ANALYSIS	**5**
3.1 System Requirements	5
3.1.1 Hardware Requirements	5
3.1.2 Software Requirements	5
3.2 User Requirements	6
3.3 Functional Requirements	6
3.4 Non-Functional Requirements	8
4. MODULES	**9**
4.1 Authentication and Profile Module	9
4.2 Guardian Management Module	9
4.3 Emergency Trigger Module	9
4.4 Emergency Detection Module	10
4.5 Alert and Communication Module	10
4.6 Guardian Dashboard Module	10
4.7 Evidence and Incident Module	10
4.8 Offline Safety Module	11
4.9 AI Module	11
4.10 Additional Safety Tools Module	11
4.11 Background Services Module	12
4.12 Settings and Permissions Module	12
5. SYSTEM ARCHITECTURE AND DATA FLOW	**13**
5.1 System Architecture Overview	13
5.2 System Architecture Diagram	14
5.3 Emergency Data Flow	15
5.4 Emergency State Machine	16
6. IMPLEMENTATION	**17**
6.1 Frontend Development	17
6.2 Security and Access Control	17
6.3 Backend and Data Layer	18
6.4 Emergency Detection and Communication	18

6.5 Android Restrictions and Legitimate Alternatives 18
7. CONCLUSION 20
APPENDIX 21
REFERENCES 30

LIST OF TABLES

NO.	TITLE	PAGE NO
3.1	Hardware Requirements	5
3.2	Software Requirements	5
6.1	Android Restrictions and Legitimate Alternatives	19

LIST OF FIGURES

NO.	TITLE	PAGE NO
5.1	System Architecture Diagram	14
5.2	Emergency Data Flow	15
5.3	Emergency State Machine	16

CHAPTER 01

INTRODUCTION

Personal safety during travel, commuting and walking alone depends on how quickly a person can signal danger and how quickly the right people can respond. Most people carry a smartphone, yet raising an alarm still takes several steps, needs the internet, and often fails when the phone is dropped, snatched or out of battery. Guardian AI addresses this by combining manual SOS triggers, automatic emergency detection, live location sharing, guardian monitoring, evidence collection and offline communication in one Android application. It adds AI features, namely route risk scoring and incident summaries, to help users and guardians make better decisions. The system prioritises reliability, safety, privacy and battery efficiency, in that order, and keeps working when internet connectivity is poor or unavailable.

1.1 OBJECTIVES

- Provide a fast SOS that can be triggered from the app, the lock screen and the power button or shake gesture within seconds.
- Detect possible emergencies automatically from phone-drop sensor patterns, low battery, route deviation and missed check-ins, with a confirmation countdown to avoid false alarms.
- Deliver alerts to guardians through more than one channel: Firebase push notification, SMS with GPS coordinates, and a local save with retry.
- Share live location, battery level and incident details with guardians who have given explicit consent.
- Collect geotagged evidence and store every emergency as an incident record.
- Keep core emergency features working with poor or no internet, using an offline assistant and cached maps.
- Add AI risk scoring, AI incident summaries and a community hotspot map.
- Protect user privacy with encrypted local storage, hashed PINs, security rules and safeguards against misuse of guardian access.
- Support English, Malayalam and Hindi, and integrate with the 112 emergency number in India.

CHAPTER 02

LITERATURE REVIEW

2.1 EXISTING SOLUTIONS

Several applications already help people share their location or call for help in an emergency. The following were reviewed from their public store listings and official descriptions .

1. bSafe
   - Features: voice-activated SOS alarm, live streaming of location and audio/video to guardians, automatic recording when SOS starts, a personal guardian network, Follow Me trip tracking, and a fake call .
   - Benefit: combines several personal safety tools in one app.
2. Noonlight
   - Features: press-and-hold SOS with a PIN to cancel, 24/7 dispatchers who contact local police, text communication, and a Safety Network that can request check-ins and see live location .
   - Note: automatic crash response, a lock-screen widget and some other features are part of paid plans .
3. Life360
   - Features: location sharing with place alerts, crash detection, SOS alerts sent to friends, family and responders, and driving reports .
   - Note: location history length, emergency dispatch and individual driver reports depend on the subscription tier .
4. 112 India (ERSS)
   - Features: India's single emergency number 112 handles calls, SMS, email, panic SOS signals and a web portal; the 112 India mobile app sends an SOS alert with location and can place a call to 112, and an SOS can also be triggered with the power button .
   - Benefit: connects the user directly to the government emergency response system, which covers police, fire, medical and disaster teams .

2.2 LIMITATIONS OF CURRENT SOLUTIONS

The applications above are useful, but a student-built system can still improve on them in the following areas. These points compare what the reviewed listings describe with what Guardian AI plans to provide.

- Single-channel dependence: The listings describe internet-based alerts, dispatch calls and live tracking. An automatic SMS fallback with GPS coordinates and a local save-and-retry queue when both internet and SMS fail is not described in them.
- False alarms and trust: Automatic detection (such as crash or drop detection) needs a confirmation step and tunable sensitivity so that false alerts do not make guardians ignore real ones.
- Cost: Several useful features, such as automatic crash response and emergency dispatch, are placed behind paid plans .
- Guardian consent and misuse: Location sharing apps can be abused for stalking, so guardian linking needs explicit consent, visibility to the user and easy revocation.
- No route risk guidance: The reviewed listings do not describe a risk score for a planned route or time of day, or a summary of an incident for guardians.
- Local fit: Support for Malayalam and Hindi, 112 integration and Indian helplines is limited in apps built mainly for other regions.
- Platform limits are rarely explained: Android limits background location and requires foreground services to show a visible notification , and power-button detection, SMS and automatic calling are also restricted (see Table 6.1). A reliable app must use legitimate alternatives and be honest about what each device can do.

2.3 PROPOSED SOLUTIONS

Guardian AI combines manual and automatic emergency triggers with a reliable multi-channel alert path and AI support. The key proposed solutions are:

- Layered emergency triggers: In-app SOS, power-button and shake detection, a lock-screen widget and a silent duress PIN, each with a confirmation countdown and cancel option.
- Automatic detection: Phone-drop detection from accelerometer and gyroscope patterns, low-battery alerts, route deviation during a safety journey, and two-way check-ins.

- Reliable delivery: Firebase Cloud Messaging and live location when online , SMS with GPS coordinates when offline, and a local record with retry when both fail.
- Consent-based guardians: Invite code or QR linking, a dashboard with status, live location and incident history, and one-tap revocation.
- AI support: A transparent route and time risk score, AI-generated incident summaries and an anonymous community hotspot map.
- Evidence and offline safety: Geotagged image capture, incident records, cached maps and a rule-based offline safety assistant.
- Regional support: English, Malayalam and Hindi, and one-tap access to 112.
- Honest platform handling: Foreground services and permission flows that follow Android rules , with two build flavors: a full-feature APK and a Play Store build with compliant fallbacks.

CHAPTER 03

SYSTEM ANALYSIS

3.1 SYSTEM REQUIREMENTS

3.1.1 HARDWARE REQUIREMENTS

Component	**Requirement**
Development computer	Intel Core i5 or equivalent processor
RAM	8 GB minimum (16 GB recommended for the Android emulator)
Hard disk	512 GB
Test device	Android smartphone with GPS, accelerometer, gyroscope, camera, microphone and a SIM card with SMS capability
Internet	Required for Firebase; the app must also work when it is unavailable

Table 3.1 : Hardware Requirements

3.1.2 SOFTWARE REQUIREMENTS

Component	**Requirement**
Language	Kotlin (Java only where required by Android libraries)
UI	Jetpack Compose
Architecture	MVVM with Clean Architecture, Hilt dependency injection, Coroutines and Flow
Local storage and background work	Room database, WorkManager, Android foreground services
Backend	Firebase Authentication, Cloud Firestore, Cloud Messaging, Cloud Storage and App Check
Maps	Google Maps SDK

Component	Requirement
AI	Gemini API for incident summaries; rule-weighted model for risk scoring
Development tools	Android Studio, Git, Firebase Emulator Suite for security-rule tests
Operating system for development	Windows, macOS or Linux

Table 3.2 : Software Requirements

3.2 USER REQUIREMENTS

- Fast SOS: The user must be able to raise an alarm within seconds without navigating through several screens.
- Automatic help: The app must detect a possible emergency and ask for confirmation, so the user does not have to act when unable to.
- Guardian choice: The user must decide who the guardians are, and be able to pause or remove them at any time.
- Works without internet: Core emergency communication must continue with poor or no internet where technically possible.
- Privacy: Location, evidence and PINs must stay private and be visible only to authorised guardians.
- Low battery use: Safety monitoring must not drain the battery.
- Safe journeys: The user must be able to share a trip and be alerted if the route is left.
- Discreet options: A silent duress PIN and a fake call must be available.
- Local language: The app must be usable in English, Malayalam and Hindi.
- Clear status: The user must always see whether monitoring, location sharing and guardians are active.
- Guardian clarity: A guardian must see who needs help, where they are and what happened, in a simple dashboard.

3.3 FUNCTIONAL REQUIREMENTS

- Authentication: The system must support registration, login and password reset with Firebase Authentication.

- Guardian linking: The system must link guardians through a one-time invite code or QR that the guardian accepts, and allow revocation.
- In-app SOS: The system must provide an SOS button with a confirmation countdown and cancel option.
- Power-button and shake SOS: The system must offer layered triggers and show which ones are active on the device.
- Duress PIN: The system must silently start the emergency workflow when the duress PIN is entered, and store PINs only as salted hashes.
- Phone-drop detection: The system must detect drops from accelerometer and gyroscope patterns and ask for confirmation before alerting.
- Battery alert: The system must warn the user at a configurable threshold and notify guardians once per battery event.
- Safety journey: The system must track a planned route, detect sustained deviation allowing for GPS error, and detect prolonged inactivity.
- Two-way check-in: The system must support timed check-ins, reminders, a grace period and guardian "Are you safe?" requests.
- Alerts: The system must send Firebase push notifications, SMS with a Google Maps link and coordinates, and retry locally when both fail.
- Emergency calling: The system must provide a Call Emergency action with 112 and always require a visible confirmation.
- Live location: The system must share location, battery and accuracy with guardians during an emergency, and stop when the emergency ends or expires.
- Guardian dashboard: The system must show status, map, battery, last update, check-ins and incident history.
- Evidence and incidents: The system must create an incident ID for every emergency, capture geotagged images and store incident records.
- AI features: The system must provide a risk score for journeys, AI incident summaries and an anonymous hotspot report map.
- Offline support: The system must provide cached maps, a rule-based offline assistant and Room storage with automatic sync.
- Fake call: The system must show a simulated incoming call after a chosen delay.

- Foreground monitoring: The system must run Guardian Mode as a foreground service with a visible notification and recover after reboot without triggering an emergency.
- Localization: The system must support English, Malayalam and Hindi.

3.4 NON-FUNCTIONAL REQUIREMENTS

The non-functional requirements describe the operating qualities of Guardian AI. The priority order is reliability, safety, privacy, battery efficiency, functionality and visual design.

- Reliability: An emergency must reach at least one channel in every failure scenario, and state must survive app kill and device reboot.
- Performance: The target is an alert sent in under 5 seconds when online and SMS dispatch in under 10 seconds when offline.
- Safety: Every automatic detection must offer a clear way to cancel wherever it is safe to do so.
- Privacy and security: Data must be protected with encrypted local storage, hashed PINs, Firestore and Storage security rules and App Check. No API keys are hard-coded and sensitive data is not logged.
- Battery efficiency: Guardian Mode should use under 4% battery per hour, using adaptive location intervals.
- Accuracy: The phone-drop false-positive rate in testing should be under 5% of detections.
- Usability and accessibility: Large touch targets, screen-reader labels, font scaling, dark mode and colour that is never the only signal.
- Maintainability: Clean Architecture with business logic in use cases and repositories, and reusable components.
- Compatibility: Behaviour must be tested on real devices because sensors, background limits and power-button handling vary by manufacturer.
- Compliance: A privacy policy, in-app data-use summary and Play policy declarations are required before a Play Store release.

CHAPTER 04

MODULES

Guardian AI is built from twelve modules. Each module has one purpose and talks to the others through the emergency state machine described in Chapter 5.

4.1 AUTHENTICATION AND PROFILE MODULE

- **Purpose :** Creates and secures the user's account and profile.
- **Features :**
    - Login, registration (name, email, password, phone number) and forgot password with Firebase Authentication.
    - Creates the Firestore user document after registration.
    - Five-page onboarding, then progressive permission requests.
    - Language and role settings (user or guardian) on the profile.

4.2 GUARDIAN MANAGEMENT MODULE

- **Purpose :** Links trusted people to the user with explicit consent.
- **Features :**
    - Generates a one-time invite code and QR valid for 24 hours.
    - Guardian sees what will be shared and accepts, creating a link with a consent timestamp.
    - User can pause or revoke a guardian at any time.
    - SMS-only contacts for people who do not have the app.

4.3 EMERGENCY TRIGGER MODULE

- **Purpose :** Lets the user start an emergency on purpose.
- **Features :**
    - Large in-app SOS button with a configurable countdown and cancel.
    - Power-button, screen-toggle, shake and lock-screen widget triggers.
    - Silent duress PIN with salted-hash storage.
    - Call Emergency button for contacts and 112.

4.4 EMERGENCY DETECTION MODULE

- **Purpose :** Detects possible emergencies automatically.
- **Features :**
    - Phone-drop detection using accelerometer and gyroscope with a 10-second confirmation.
    - Battery-low alert with a configurable threshold.
    - Safety journey with route deviation and inactivity detection.
    - Two-way check-in with reminders and a grace period.
    - Smart false-alarm filter that learns the user's normal motion.

4.5 ALERT AND COMMUNICATION MODULE

- **Purpose :** Gets the alert to guardians through whichever channel works.
- **Features :**
    - Firebase Cloud Messaging push notifications.
    - SMS fallback with a Google Maps link and GPS coordinates, in English, Malayalam or Hindi.
    - Local save and WorkManager retry when internet and SMS both fail.
    - Live location sharing with status "Emergency location sharing active".

4.6 GUARDIAN DASHBOARD MODULE

- **Purpose :** Shows guardians what is happening to the people they protect.
- **Features :**
    - Status: Safe, Safety Journey Active, Emergency, Offline or Low Battery.
    - Live map, battery percentage and last update time.
    - Active check-ins and an "Are you safe?" request.
    - Incident history with trigger type, evidence and AI summary.

4.7 EVIDENCE AND INCIDENT MODULE

- **Purpose :** Records what happened during an emergency.
- **Features :**
    - Incident IDs in the form INC-2026-00001.

- Geotagged image capture with a visible capture indicator.
- Live video prototype with a Start and Stop control, designed so WebRTC can replace it later.
- Firestore for metadata, Firebase Storage for images and video, and configurable retention.

4.8 OFFLINE SAFETY MODULE

- **Purpose :** Keeps the app useful with poor or no internet.
- **Features :**
  - Cached maps and last known location.
  - Rule-based offline emergency assistant with an SOS button.
  - Room storage for incidents, pending alerts and settings.
  - Work Manager sync when the network returns.

4.9 AI MODULE

- **Purpose :** Adds prediction and explanation on top of the safety workflow.
- **Features :**
  - Risk score from 0 to 100 for a route and time, with reasons and a safer route suggestion.
  - AI incident summary for guardians, labelled as AI-generated, with a template fallback.
  - Anonymous community hotspot reports shown as a heatmap that feeds the risk score.

4.10 ADDITIONAL SAFETY TOOLS MODULE

- **Purpose :** Gives the user discreet ways to leave an unsafe situation.
- **Features :**
  - Fake incoming call with a chosen caller name, number and delay.
  - Share location quick action.
  - Guardian Mode status on the home screen.

4.11 BACKGROUND SERVICES MODULE

- **Purpose :** Keeps monitoring running safely in the background.
- **Features :**
    - Foreground service for sensors, location, journeys and emergency state, with a visible notification.
    - Boot receiver that restores settings and never triggers an emergency by itself.
    - Adaptive location intervals to save battery.

4.12 SETTINGS AND PERMISSIONS MODULE

- **Purpose :** Lets the user configure safety features and control access.
- **Features :**
    - Settings for emergency, detection, AI, evidence, location, offline and language.
    - Progressive permission requests with an explanation and a fallback for each denial.
    - Shows which trigger layers are active on the device.

CHAPTER 05

SYSTEM ARCHITECTURE AND DATA FLOW

5.1 SYSTEM ARCHITECTURE OVERVIEW

The architecture of Guardian AI is divided into four layers, built with Kotlin, MVVM and Clean Architecture.

1. Presentation Layer (The Screens)

This layer holds the Jetpack Compose screens for login, onboarding, home with SOS, journey, guardians, incidents and settings, and the guardian dashboard. Screens only show state and send user actions to view models. No business logic lives here.

2. Domain Layer (The Rules)

This layer holds the models, repository interfaces and use cases, such as start SOS, evaluate drop, check route deviation and score risk. It does not depend on Android or Firebase, so it can be unit tested.

3. Data Layer (The Storage)

This layer implements the repositories. Room stores incidents, pending alerts, locations and settings on the device. Firebase Authentication, Firestore, Cloud Messaging and Storage hold accounts, guardian links, incident records and media in the cloud. WorkManager syncs the two when the network returns.

4. Emergency Engine and Services (The Response)

This layer contains detection, triggers, communication and the central emergency state machine, and runs inside an Android foreground service. Every trigger, whether manual or automatic, enters the same state machine, so the response is identical whatever starts it.

5.2 SYSTEM ARCHITECTURE DIAGRAM

The system architecture diagram shows the components of Guardian AI and how an alert reaches a guardian.

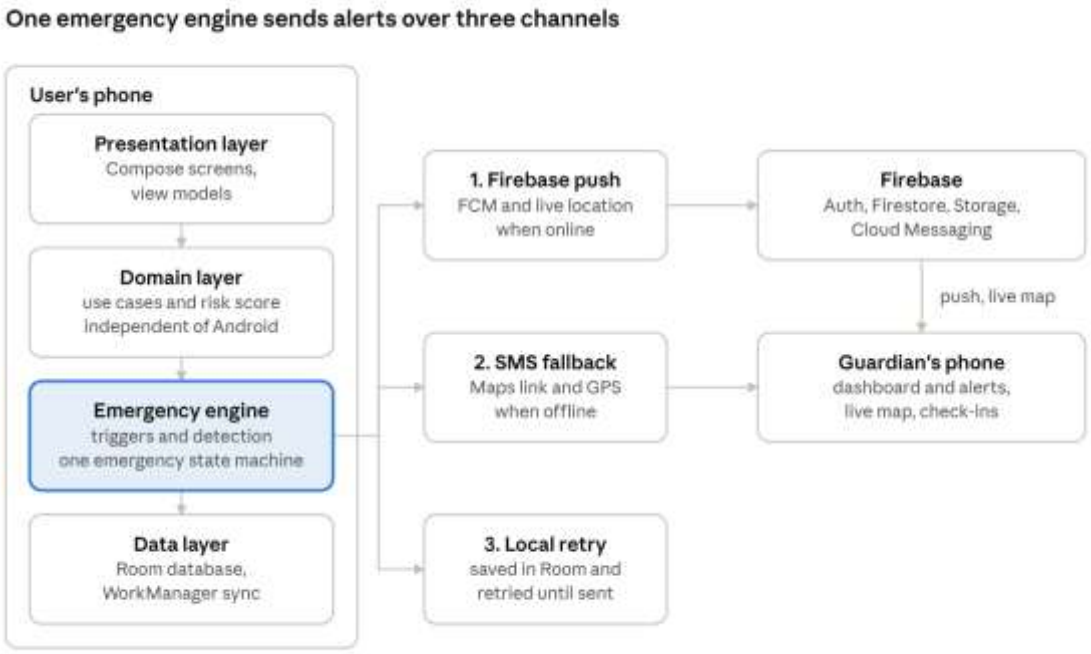


Fig 5.1 : System Architecture Diagram

Every trigger enters the emergency engine, which sends the alert through Firebase when online, by SMS when offline, and into a local retry queue if both fail.

- The figure shows Guardian AI in layers: the user interface, the presentation logic, the domain logic, the data layer, and the background services.
- The interface is built with Kotlin and Jetpack Compose. Each screen talks to a ViewModel (MVVM), so the screens hold no business logic.
- The domain layer holds the emergency rules: trigger handling, the countdown, alert sending and risk scoring. It does not depend on Android classes, so it can be tested separately.
- The data layer has two parts. Room stores data on the phone so the app works without internet. Firebase (Authentication, Firestore, Cloud Messaging, Storage) holds shared data and sends push alerts.
- A foreground service with a visible notification keeps detection running in the background. WorkManager retries any alert that could not be sent.

- External services are the Google Maps SDK for location and maps, and the Gemini API for incident summaries. Hilt connects the layers, so each can be replaced or tested alone.
- Because every trigger goes through the same layers, a manual SOS and an automatic detection are handled the same way.

5.3 EMERGENCY DATA FLOW

The data flow shows what happens from a trigger until the guardian is notified, including the fallback channels.



Fig 5.2 : Emergency Data Flow

After the countdown the incident is saved on the device first. Firebase is tried when the internet is on, SMS when it is off, and a local retry queue catches a failed SMS.

A trigger such as the SOS button, a phone drop, a low battery or a route deviation goes through a cancellable confirmation countdown, and then the app collects the location, battery level and time. The alert is sent by push notification when online, by SMS with a Google Maps link when offline, and saved locally for retry if both fail.

5.4 EMERGENCY STATE MACHINE

A single state machine controls every emergency and is the source of truth for its status. Its state is saved to Room at once, so it survives an app kill or a reboot.

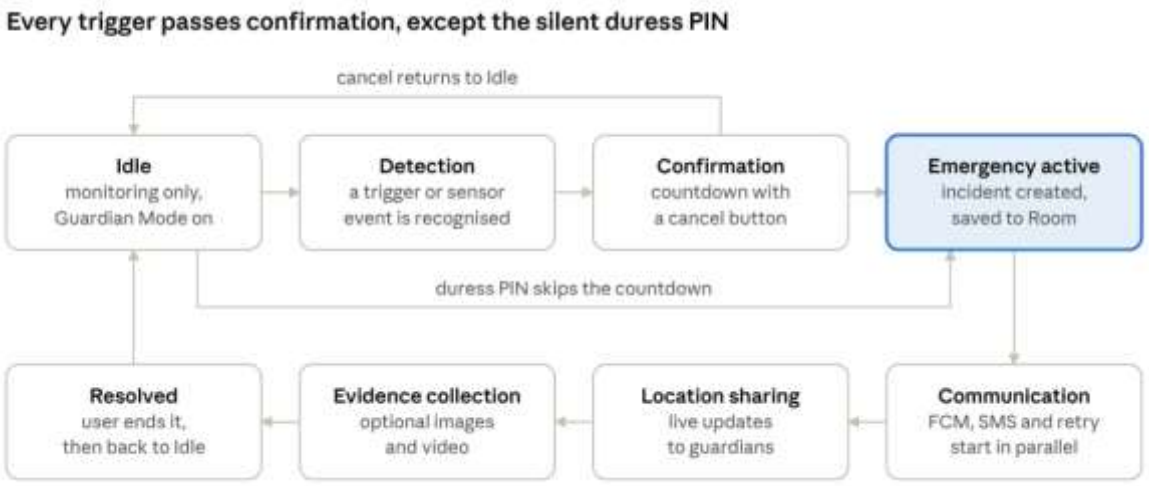


Fig 5.3 : Emergency State Machine

- One state machine controls every emergency, so all triggers follow the same path through eight states: Idle, Detection, Confirmation, Emergency Active, Communication, Location Sharing, Evidence Collection and Resolved.
- A sensor or button event moves the app from Idle to Detection, and then to Confirmation, where the user can cancel and return to Idle without any alert being sent.
- When the countdown ends or the user confirms, the app enters Emergency Active, and the duress PIN goes straight to this state without a countdown.
- Emergency Active starts three activities together: Communication sends the alerts, Location Sharing streams the location, and Evidence Collection captures photos.
- The user or a guardian closes the incident, which moves the app to Resolved and stops all activities.
- The current state is saved on the phone, so the emergency continues even if the app is killed or the phone restarts.
- Because only defined transitions are allowed, every trigger behaves the same way and the app cannot enter an invalid state.

CHAPTER 06

IMPLEMENTATION

This chapter describes how Guardian AI will be built. The work is planned in eight phases, and each phase is built, tested on a real device and verified before the next one starts.

6.1 FRONTEND DEVELOPMENT

- The app is written in Kotlin with Jetpack Compose and a bottom navigation bar with five tabs: Home, Journey, Guardians, Incidents and Settings.
- Each screen has a view model that exposes state, and screens contain no business logic.
- Dependency injection with Hilt keeps use cases and repositories separate from the interface.
- The SOS button is always one tap from the home screen and is also available from the lock screen and the offline assistant.
- Strings live in Android resource files, so English, Malayalam and Hindi can be switched without code changes.
- The visual design uses one clear identity (a small palette and one font pair), large emergency controls, high contrast, screen-reader labels and dark mode.
- Every button either works or shows "Coming Soon".

6.2 SECURITY AND ACCESS CONTROL

- Firebase Authentication identifies users, and Firebase App Check blocks clients that are not the app.
- Firestore and Storage security rules allow a user to read and write only their own data, and a guardian to read only users who accepted the invite and have not revoked it.
- Sensitive local data uses encrypted storage. PINs and the duress PIN are stored only as salted hashes.
- API keys are not hard-coded, and sensitive data is never written to logs.
- Hotspot reports are anonymous, location-fuzzed and rate-limited.
- Anti-misuse safeguards: guardian links need the user's invite and consent, the user is told when a guardian views live location, and revocation takes one tap.
- Incident and media retention is configurable, with scheduled cleanup.

6.3 BACKEND AND DATA LAYER

- Firestore stores users, guardian links, journeys, incidents, check-ins, locations, invites and hotspot reports. Firebase Storage stores incident images and video.
- Firebase Cloud Messaging delivers alerts for SOS, phone drop, route deviation, low battery, missed check-in, and session start and end .
- Room stores users, guardians, journeys, incidents, locations, check-ins, evidence, pending alerts and offline assistant content on the device.
- WorkManager syncs pending data with exponential backoff. For incident facts the device record wins, and server-set fields such as acknowledgements are merged.
- Gemini is called through a backend or a restricted key to create incident summaries, with a template fallback when the model is unavailable.

6.4 EMERGENCY DETECTION AND COMMUNICATION

- Phone drop: a free-fall reading, then an impact spike, then stillness or abnormal rotation, checked against the user's motion baseline. A 10-second confirmation screen offers I'm Safe and Send SOS, and no response starts the emergency.
- Route deviation: distance from the planned route is compared against GPS accuracy plus a buffer (default 150 m) sustained for a time window (default 60 s), so small errors do not raise alerts.
- Battery: one alert per low-battery event, reset only after the battery recovers.
- Check-in: a deadline, then a reminder, then a grace period, then a guardian notification.
- Risk score: a transparent rule-weighted score from time of day, hotspot reports, route isolation and past incidents, with the top reasons shown to the user.
- Communication order: Firebase and live location first, SMS with a Google Maps link and coordinates second, and a Room record with WorkManager retry last. Channels run in parallel where possible, and the status of each is stored on the incident.
- 112: the Call Emergency action opens the dialer with the incident location ready. Calls always follow a visible confirmation or countdown, and no call is made silently.

6.5 ANDROID RESTRICTIONS AND LEGITIMATE ALTERNATIVES

No feature will bypass Android security. Where Android does not allow a capability, the closest legitimate alternative is implemented and the limit is documented .

Feature	Restriction	Implementation in Guardian AI
Power-button SOS	No public API for hardware power-key events	Layered: accessibility key events (APK build), screen on/off counter in the foreground service, shake gesture and a lock-screen widget
Background location	Needs foreground then background permission, and Google Play restricts it to core features	Foreground service with a visible notification, and a separate background-location request with a clear explanation
Sending SMS	SMS is a restricted permission on Google Play	APK build sends directly; the Play build opens a prefilled SMS the user confirms, with Firebase push as the main channel
Automatic calling	Calling permission is sensitive	A confirmation screen first, then the call; never silent
Foreground services	Must show a visible notification and declare a type	Persistent "Guardian AI is protecting you" notification with Open and Stop actions
Boot receiver	Some services cannot start directly from boot	Restore settings and schedule work on boot, and ask the user to reopen the app if needed; never auto-trigger an emergency
Camera and microphone in the background	Not allowed without visible indicators	Evidence capture runs only while an emergency foreground service shows an active-capture notification
Battery optimisation	Manufacturers may stop background services	In-app guide to exempt the app, and WorkManager recovery

Table 6.1 : Android Restrictions and Legitimate Alternatives

The rule is that if a capability is not possible, the app says so and offers the fallback. A feature never appears to work when it does not.

CHAPTER 07
CONCLUSION

Phase 1 established Guardian AI as a practical safety application rather than a user interface demonstration. The problem of unsafe travel and delayed assistance was studied, existing applications such as bSafe, Noonlight, Life360 and 112 India were reviewed, and development gaps were identified. The requirements, twelve modules, layered architecture, data model, security measures and Android restrictions were analysed. A phased development plan with clear exit gates was also prepared.

Reliability is the primary design objective. All manual and automatic triggers enter a common emergency state machine, ensuring a consistent response. Triggers such as phone drops, low battery, route deviation and missed check-ins initiate a confirmation countdown with a cancellation option to reduce false alarms. Alerts use Firebase push notifications when online, SMS with GPS coordinates when offline, and a local save-and-retry queue if both channels fail.

Privacy and consent are essential requirements. Guardians are connected only through accepted invitations, and users can pause or remove access at any time. Live location is shared only during journeys or emergencies. PINs are stored as salted hashes, local data is encrypted, and Firebase security rules restrict guardians to authorised users, preventing misuse of location tracking.

AI features enhance the safety workflow through transparent route-risk scoring, AI-generated incident summaries and anonymous community hotspot reports. However, AI never overrides manual SOS or confirmation countdowns. The offline assistant provides local, rule-based emergency guidance without requiring cloud access.

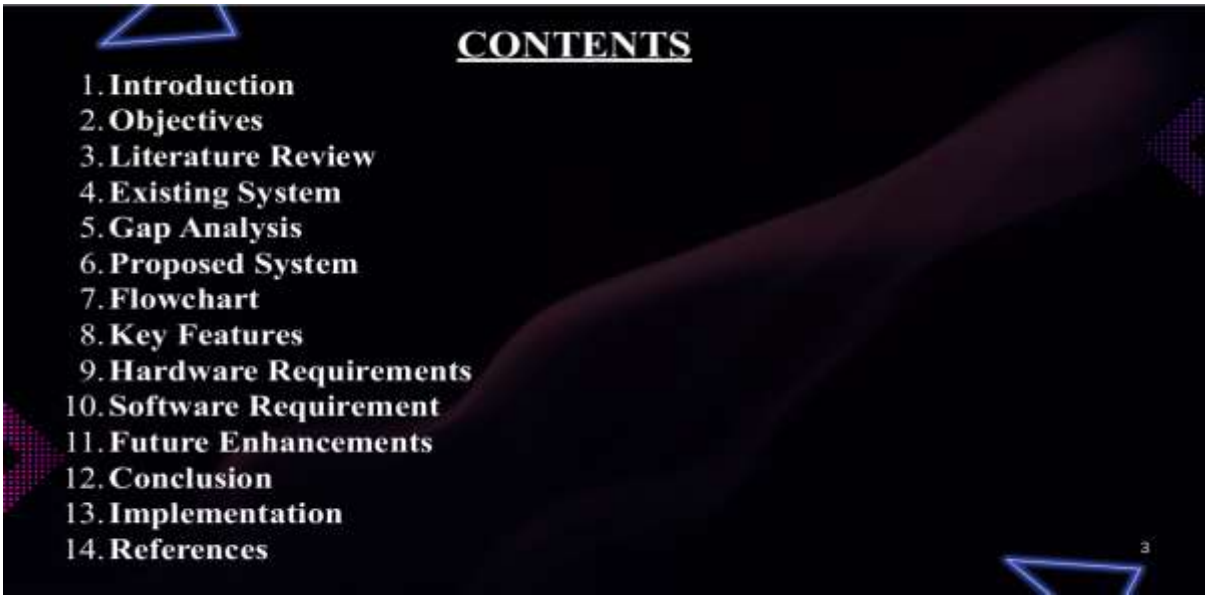
Android restrictions on power-button detection, background location, silent SMS and automatic calling were considered. Guardian AI will use legitimate alternatives, including layered triggers, a foreground service with visible notifications and compliant fallbacks. Device capabilities and limitations will be clearly communicated to users.

With the analysis, architecture and development plan completed, Guardian AI is ready for implementation, beginning with the foundation and emergency core. Each phase will be developed, tested on a real device and verified before proceeding.

APPENDIX


GUARDIAN AI
AI-Powered Personal Safety & Emergency Intelligence
1


Guardian AI – AI-Powered Personal Safety & Emergency Intelligence application
Team Members
AKSHAY BA(LMC23CS026)
ANANDHU A(LMC23CS035)
MANU VS(LMC23CS085)
Project Guide
Ms.NISHA O.S
Assistant Professor
DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
2


CONTENTS
1. Introduction
2. Objectives
3. Literature Review
4. Existing System
5. Gap Analysis
6. Proposed System
7. Flowchart
8. Key Features
9. Hardware Requirements
10. Software Requirement
11. Future Enhancements
12. Conclusion
13. Implementation
14. References
3

INTRODUCTION

- Personal safety has become a major concern due to the increasing number of accidents, crimes, and medical emergencies in today's society.
- Existing emergency applications mainly rely on manual user intervention, which can delay emergency response during critical situations.
- Guardian AI is an AI-powered mobile safety application designed to provide intelligent monitoring, rapid emergency assistance, and continuous user protection.

4

INTRODUCTION(Contd.)

- The system integrates artificial intelligence, GPS tracking, smartphone sensors, and cloud communication to improve personal safety.
- Although several personal safety applications are available, most existing solutions depend heavily on manual SOS activation and continuous internet connectivity, which may reduce their effectiveness during critical situations.

5

Objectives

- To develop an AI-powered personal safety application that provides intelligent emergency detection and immediate assistance during accidents, crimes, and medical emergencies.
- To enable rapid emergency communication by sharing live location, emergency alerts, battery status, and other critical information with trusted guardians and emergency contacts.
- To improve user safety through advanced features such as automatic emergency detection, offline emergency support, live video streaming, and secure evidence collection.

6

Objectives(Contd.)

- To provide a reliable and user-friendly safety platform by integrating Artificial Intelligence, GPS tracking, smartphone sensors, and cloud communication technologies.

- To enhance situational awareness and reduce emergency response time by delivering proactive protection and continuous guardian monitoring during critical situations.

7

Literature Review

Paper Name	Authors / Year	Key Findings
SafeHer: A Mobile-Based Women's Safety Platform with Real-Time SOS, Offline SMS Alerts, and Community Support	Y. Kanse, S. Joshi, R. Jadhav, H. Hinukale, P. Charankar & N. D. Gaikwad (2026)	Developed a mobile safety platform combining one-tap SOS, live GPS sharing, offline SMS alerts, shake-based triggering, and fake-call functionality.
Empowering Personal Safety Through Mobile Apps (chapter in AI Tools and Applications for Women's Safety)	S. Banerjee et al. (2024)	Reviews safety applications such as bSafe and Life360, highlighting dependence on continuous location tracking and manual SOS activation.
Overall Research Gap		Existing research has individually explored motion-sensor-based detection, audio-based AI classification, GPS tracking, SOS systems, and emergency communication.

8

Existing System

- Current safety applications provide basic features such as SOS alerts, live location sharing, and emergency contact notification.

- Most applications require users to manually trigger emergency alerts, making them less effective in unexpected situations.

- Many existing systems depend heavily on internet connectivity for communication and location tracking.

9

Gap Analysis

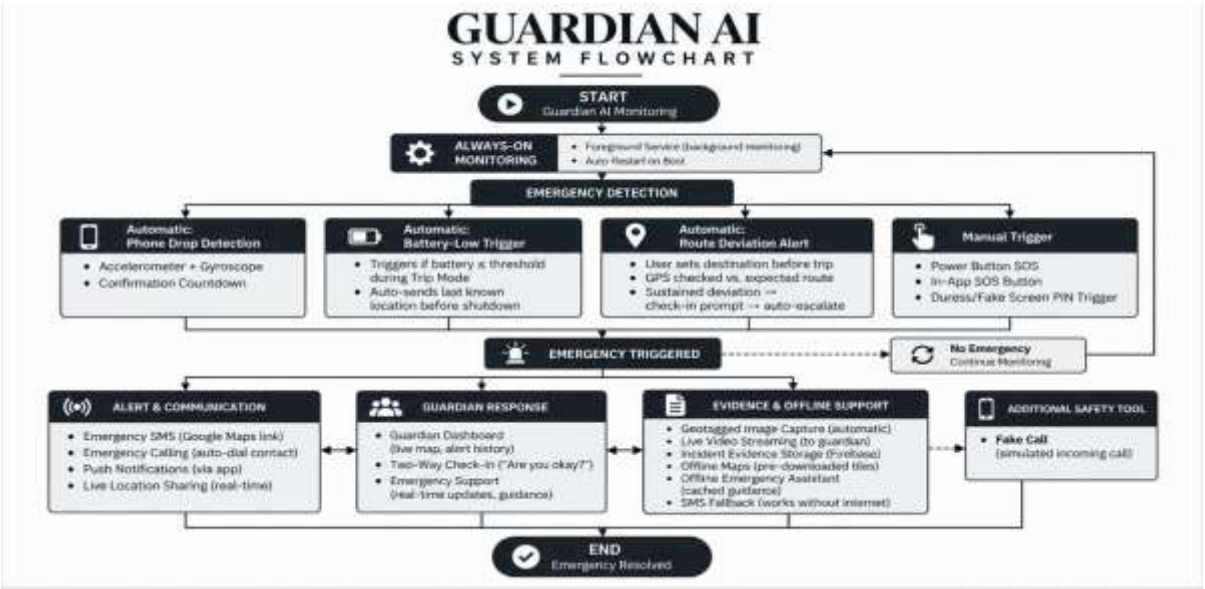
- Current safety applications do not provide intelligent mechanisms to automatically detect emergencies without requiring user interaction.
- Most existing systems lack integrated offline assistance, automatic evidence collection, and real-time monitoring for trusted guardians.
- There is a need for an AI-powered platform that combines intelligent detection, emergency communication, and continuous user protection within a single application.

10

Proposed System

- Guardian AI is an intelligent mobile-based personal safety application that utilizes artificial intelligence to provide proactive emergency detection and rapid response.
- The system automatically detects emergency situations using smartphone sensors and instantly shares live location, emergency alerts, and critical information with trusted guardians.
- By integrating AI, GPS tracking, cloud communication, and offline emergency support, Guardian AI delivers a comprehensive and reliable personal safety solution.

11




Key Features
Emergency Detection
Phone Drop Detection
Battery-Low Trigger
Route Deviation Alert
Emergency Trigger
Power Button SOS
In-App SOS Button
Fake Screen PIN Trigger
Alert & Communication
Emergency SMS
Emergency Calling
Live Location Sharing
Push Notifications
13


Key Features(Contd.)
Guardian-Side Features
Guardian Dashboard
Two-Way Check-In
Evidence & Documentation
Geotagged Image Capture
Live Video Streaming
Incident Evidence Storage
Offline Support
Offline Maps
Offline Emergency Assistant
SMS Fallback
14


Key Features(Contd.)
Additional Safety Tools
Fake Call
System-Level
Always-On
Foreground Service
Auto-Restart on Boot
15

Hardware Requirements

- Android smartphone equipped with GPS, camera, microphone, accelerometer, gyroscope, and stable internet connectivity.
- Quad-Core processor with a minimum of 4 GB RAM and adequate internal storage for smooth application performance.
- Sufficient battery capacity and required hardware sensors to support continuous monitoring and emergency detection.

16

Software Requirements

- Android Studio as the primary development environment with Java or Kotlin for Android application development.
- Firebase Authentication, Firestore Database, and Cloud Storage for secure user management and cloud-based data storage.
- Google Maps API, CameraX, SMS Manager, and Android Location Services to implement navigation, communication, and emergency functionalities.

17

Future Enhancements

- Integrate advanced AI algorithms capable of predicting potential threats and improving emergency decision-making.
- Extend support to wearable devices, smartwatches, and community-based emergency assistance for enhanced user protection.
- Introduce predictive analytics, verified transportation recommendations, and cloud-based intelligent safety services.

18

Conclusion

- Guardian AI combines artificial intelligence with modern mobile technologies to deliver a comprehensive and intelligent personal safety solution.
- The application integrates emergency detection, communication, evidence collection, and guardian monitoring into a single unified platform.

19

Conclusion(Contd.)

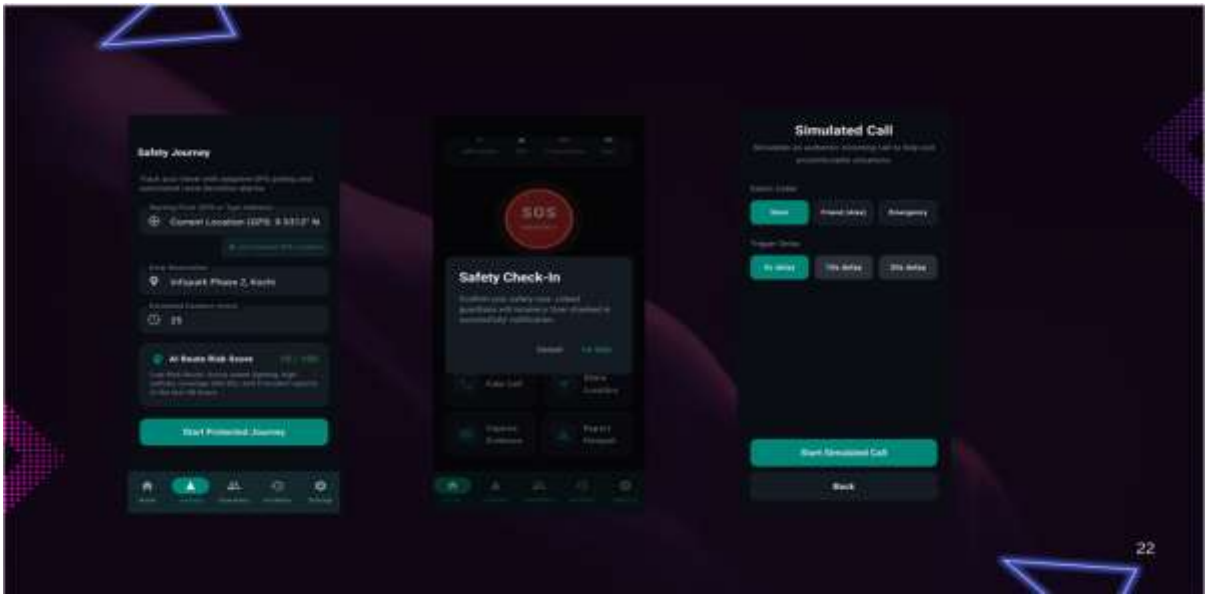
- It securely maintains emergency records, evidence, and incident history that can support future investigations and emergency response.
- By improving emergency response time and providing proactive protection, Guardian AI enhances user safety and contributes to a more secure society.

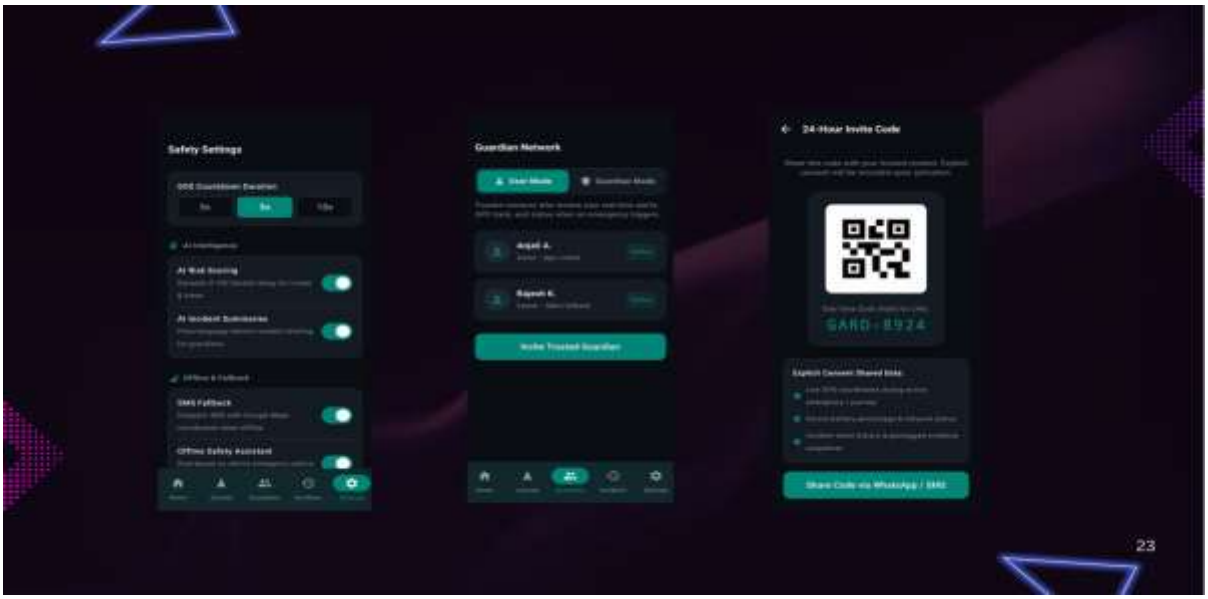
20

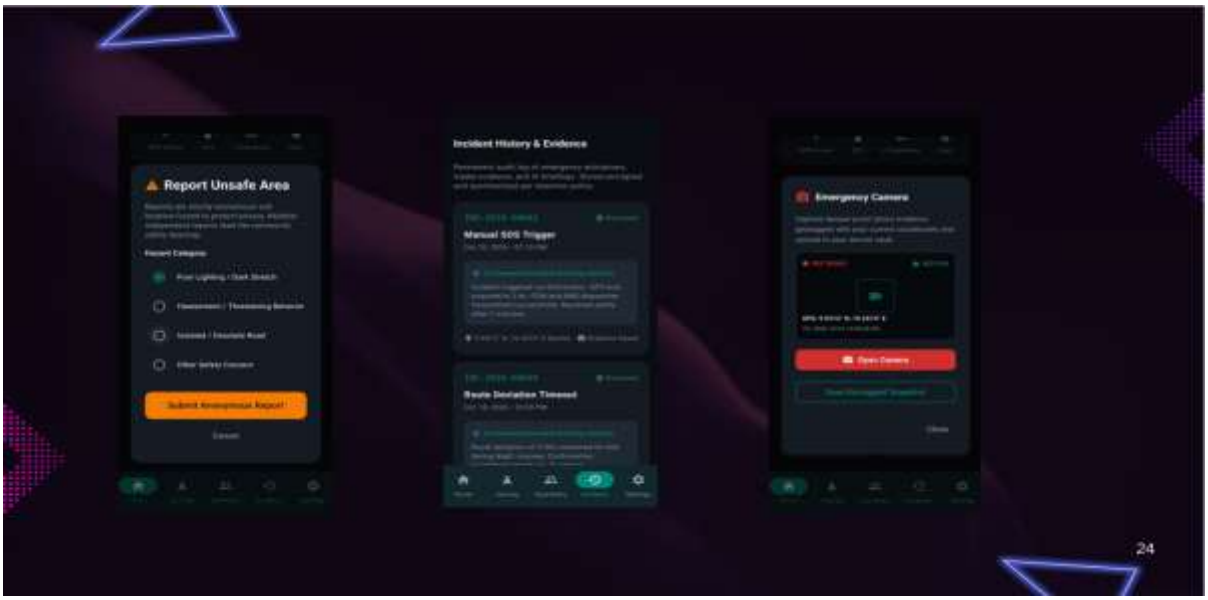
Implementation

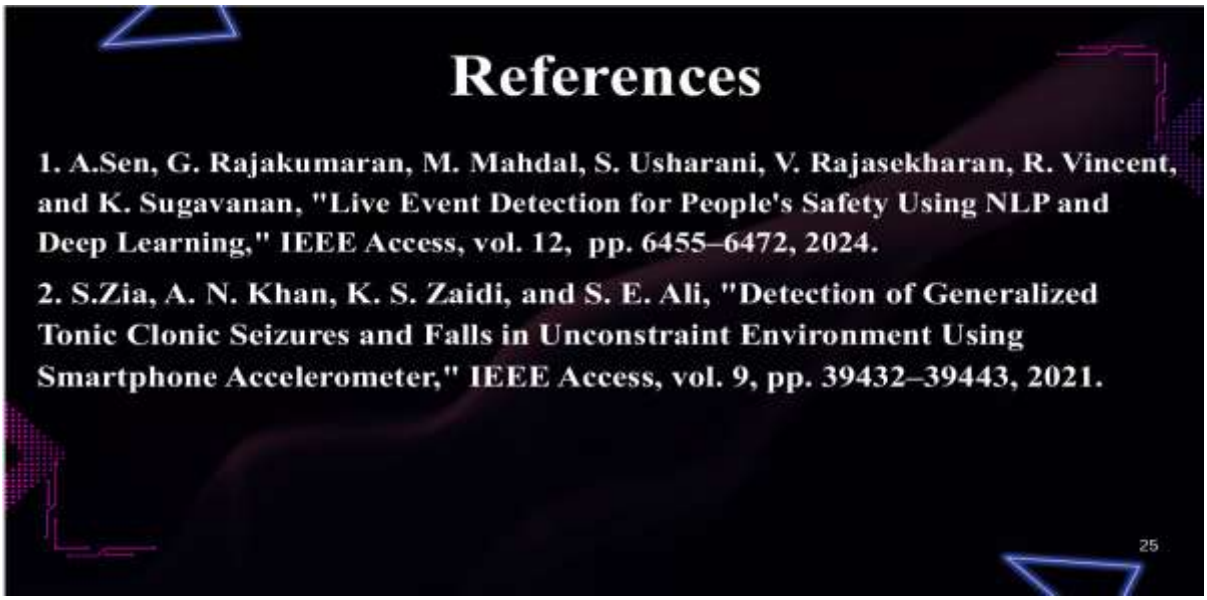


21


Safety Journey
Safety Check-In
Simulated Call
22


Safety Settings
Guardian Network
24-Hour Invite Code
23


Report Unsafe Area
Incident History & Evidence
Emergency Camera
24


References
1. A.Sen, G. Rajakumaran, M. Mahdal, S. Usharani, V. Rajasekharan, R. Vincent, and K. Sugavanan, "Live Event Detection for People's Safety Using NLP and Deep Learning," IEEE Access, vol. 12, pp. 6455–6472, 2024.
2. S.Zia, A. N. Khan, K. S. Zaidi, and S. E. Ali, "Detection of Generalized Tonic Clonic Seizures and Falls in Unconstraint Environment Using Smartphone Accelerometer," IEEE Access, vol. 9, pp. 39432–39443, 2021.
25

REFERENCES

[1] Live Event Detection for People's Safety Using NLP and Deep Learning – https://doi.org/10.1109/ACCESS.2023.3349097

[2] Detection of Generalized Tonic Clonic Seizures and Falls in Unconstraint Environment Using Smartphone Accelerometer – https://doi.org/10.1109/access.2021.3063765

[3] SafeHer: A Mobile-Based Women's Safety Platform with Real-Time SOS, Offline SMS Alerts, and Community Support – https://www.ijert.org/safeher-a-mobile-based-women-s-safety-platform-with-real-time-sos-offline-sms-alerts-and-community-support-ijertv15is051542

[4] Empowering Personal Safety Through Mobile Apps – https://www.igi-global.com/chapter/empowering-personal-safety-through-mobile-apps/337757

[5] Foreground services overview – https://developer.android.com/develop/background-work/services/fgs

[6] Firebase Cloud Messaging – https://firebase.google.com/docs/cloud-messaging

[7] Save data in a local database using Room – https://developer.android.com/training/data-storage/room.

[8] WorkManager (API reference) – https://developer.android.com/reference/androidx/work/WorkManager